

- 1) write an algorithm called Find-largest that finds the largest number in an array using a divide and conquer strategy

Algorithm

Algorithm Find-largest (A, low, high)

Input: Array A[], low, high

output: largest element in the array

- 1) IF low = high
 return A[low]
- 2) mid = (low + high) / 2
- 3) leftMax = Find-largest (A, low, mid)
- 4) rightMax = Find-largest (A, mid + 1, high)
- 5) IF leftMax > rightMax
 return leftMax
 Else
 return rightMax

Example

Array = {50, 40, -5, -9, 45, 90, 65, 85, 75}

- Divide into two halves repeatedly
- left half maximum = 50
- Right half maximum = 90
- largest element = 90

Time complexity

- Recurrence Relation:
$$T(n) = 2T(n/2) + O(1)$$
- Using Master theorem:
time complexity = $O(n)$
- space complexity:
space complexity = $O(n)$

2) Analyze how to print all the Prime in the single linked list having a list of 24 → 21 → 57 → 31 → 12 → 47 → 7 → 38 →

linked list

24 → 21 → 57 → 31 → 12 → 47 → 7 → 38 → Null

Algorithm

algorithm printprime(head)

1) $ptr = head$

2) while $ptr \neq NULL$

$num = ptr \rightarrow data$

 if $isprime(num)$

 Print num

$ptr = ptr \rightarrow next$

Algorithm $isprime(n)$

1) if $n < 2$

 return false

2) for $i = 2$ to $\sqrt{n}$

 if $n \% i == 0$

 return false

3) Return True

No de

24

21

57

31

prime ?

No

No

No

Yes

12

NO

47

yes

7

yes

38

NO

Output

prime numbers: 31 47 7

Time complexity

- Traversing linked list = $O(n)$
- Prime checking for each node = $O(\sqrt{m})$
- overall complexity = $O(n\sqrt{m})$

Space complexity

- $O(1)$ constant extra space